

ORDER & TRANSACTION

Code Enhancement & Refactoring Report

Sabeka Backend · src/app/api/Order/ · src/app/api/Transaction/

Generated April 29, 2026

35

Files Reviewed

32

Issues Found

7

Critical Issues

14

High Issues

11

Medium Issues

ORDER DOMAIN

20 files · 18 issues · 4 critical

TRANSACTION DOMAIN

15 files · 14 issues · 3 critical

Goal: Clean code · Clean structure · Readable · Maintainable · Production-safe

NAVIGATION

Table of Contents

1	Issue Catalog — At a Glance
2	Current Architecture Overview
3	Order Domain — Deep Dive
3.1	God Function Problem (add.service.ts)
3.2	Production-Unsafe Logging
3.3	Commented-Out Dead Code
3.4	God Repository Create (50-line inline type)
3.5	MySQL-Specific Raw SQL
3.6	Redundant Count Query
4	Transaction Domain — Deep Dive
4.1	Typo in Class Name
4.2	Parameter Reassignment with var
4.3	Fragile JSON Parsing
4.4	Cache Invalidation Inside Data Layer
4.5	Ambiguous Balance Field
4.6	Ambiguous Withdraw Validation
4.7	MySQL-Specific Raw SQL
5	Clean Target Folder Structure
6	Refactored Code Examples
7	Cross-Cutting Improvements
8	Priority Action Plan (3 Sprints)
9	Before vs After Scorecard

SECTION 1

Issue Catalog — At a Glance

Critical Issues — Fix Now

ID	Location	Issue
C1	Order/services/add.ts	God function — 476 lines mixing 5 responsibilities
C2	Order/utis/calculateOrder.ts	console.log() + fs.appendFileSync() in production code path
C3	Transaction/repository/index.ts	Cache invalidation (redisClient.del) inside the data layer
C4	Transaction/services/withdraw.ts	Balance check logic is ambiguous — may over-allow withdrawals

High Priority — Fix This Sprint

ID	Location	Issue
H1	Transaction/repository/index.ts	Typo: TransactionRepoistory (misspelled class name)
H2	Transaction/repository/index.ts	var lang = lang re-assignment inside userTransaction()
H3	Order/repository/index.ts	create() parameter is a 50+ line inline anonymous type
H4	Order/repository/index.ts	MySQL-only raw SQL: IF(), MONTH(), UTC_TIMESTAMP()
H5	Transaction/repository/index.ts	MySQL-only raw SQL: MONTH(created), CAST(...)
H6	Transaction/repository/index.ts	JSON.parse(type '{}') fragile JSON parsing (x3)
H7	Order/utis/calculateOrder.ts	50+ lines of large commented-out dead code block
H8	Order/repository/pendingOrder.ts	UTC_TIMESTAMP() is MySQL-specific, not portable

Medium Priority — Next Sprint

ID	Location	Issue
M1	All services	trx?: any — no proper Knex.Transaction type
M2	All repositories	PageCount (capital C) — inconsistent naming convention
M3	Transaction/services/currentBalance.ts	availWithdraw silently aliases availBalance (same field)
M4	Order/services/add.ts	var used instead of let/const (5 occurrences)
M5	Order/repository/index.ts	Two separate DB count queries that can be one
M6	All files	No shared response type — each service returns different shape

SECTION 2

Current Architecture Overview

The existing code follows a Controller → Service → Repository pattern, but the implementation has drifted: services have grown into god functions, repositories mix concerns, and raw MySQL SQL is scattered throughout.

Controller	Thin HTTP adapter — reads req.body/res.locals. Mostly good ■ — keep as-is
Service	476-line god functions mixing calculation, DB calls, HTTP calls, file system writes, and email triggers. ■ NEEDS MAJOR REFACTOR
Repository	Mixes cache invalidation, raw SQL, and DB queries. Contains typos and MySQL-specific syntax. ■ NEEDS CLEANUP
ORM (Knex/Objection)	MySQL ORM — raw SQL calls with MySQL-only functions. ■ SQL NEEDS PORTING

SECTION 3

Order Domain — Deep Dive

3.1 — The God Function Problem (services/add.ts)

The `addService()` function is 476 lines long and mixes 6 separate responsibilities into a single try/catch block. This makes it impossible to test, difficult to read, and dangerous to change.

#	Responsibility	Should be
1	Cart validation & silver-active check	<code>validateOrderPreconditions()</code>
2	Order calculation	<code>calculateOrder()</code> – already separate, keep it
3	Voucher validation & discount calculation	<code>applyVoucherDiscount()</code>
4a	VISA payment path → CyberSource → pendingOrder	<code>handleVisaPaymentPath()</code>
4b	WALLET payment path → create order + transaction	<code>handleWalletPaymentPath()</code>
5	Email notification (fire-and-forget)	<code>notifyOrderCreated()</code> – post-commit
6	Log writing to flat file	Replace with <code>logger.info()</code> – remove <code>fs.appendFileSync</code>

BEFORE — Current Code

```
// services/add.ts — CURRENT (476 lines)
export const addService = async ({ ... }) => {
  let total: any = 0; // 'any' for a financial
  let taxes: any = 0;
  let transaction_fees: any = 0;
  try {
    // 1. Email check (10 lines)
    // 2. Cart query (20 lines)
    var { details, subTotal } = await calculateOrder()
    var discount = 0; // var
    // 4. Visa path (70 lines)
    // 5. Wallet create (40 lines)
    fs.appendFileSync('logs/orders.log', ...) //
    try { userEvents.publish('send-invoice-email', {
    catch (error) { console.log('error in send email')
  } catch (error) {
    await trx.rollback();
    throw error;
  }
};
```

AFTER — Clean Code

```
// services/add.ts — CLEAN (orchestrator only)
export const addService = async (params: AddOrderParams)
: Promise<AddOrderResult> => {
  const trx = await Model.startTransaction();
  try {
    await validateOrderPreconditions(params);
    const cart = await loadUserCart(params.userId, trx);
    const calculation = await calculateOrder({ ...params, cart,
    const pricing = await applyVoucherDiscount(params, calc
    const total = computeTotal(pricing);

    let result: AddOrderResult;
    if (params.paymentMethod === 'visa' && total > 0) {
      result = await handleVisaPaymentPath({ ...params, total,
    } else {
      result = await handleWalletPaymentPath({ ...params, total
    }
    await trx.commit();
    return result;
  } catch (error) {
    await trx.rollback();
    logOrderError(error, params); // structured logger
    throw error;
  }
};
```

3.2 — Production-Unsafe Logging (utils/calculateOrder.ts)

Two patterns block the Node.js event loop and leak inventory data to unstructured plain-text files in production. The project already has a **logger** from `.././././core` — use it everywhere.

BEFORE — Current Code

```
// calculateOrder.ts — CURRENT
fs.appendFileSync(
  'logs/orders.log',
  `userCartTotals: ${JSON.stringify(userCartTotals)}\n`
); // ■ synchronous disk write — blocks event loop

console.log('fractionBar ', fractionBar); // ■
```

AFTER — Clean Code

```
// calculateOrder.ts — CLEAN
import { logger } from '../.../core';

// DEBUG only — never emits in production
logger.debug('Cart totals computed', { userCartTotals });

logger.debug('Processing fraction bar',
  { fractionBarId: fractionBar.id });
```

3.3 — Large Commented-Out Dead Code (utils/calculateOrder.ts)

A 50+ line commented-out block sits at lines 42–67. Dead code belongs in **git history**, not in source files. It adds cognitive load and confuses future readers about what is actually active.

Rule

If code is commented out and not planned for reactivation in the current sprint — **delete it**. Git preserves every prior state. You can always recover it with git log.

3.4 — God Repository create() (repository/index.ts)

The **create()** method accepts a 50+ line anonymous inline type in its signature. This makes the method unreadable and cannot be reused or tested without copying the entire inline type.

BEFORE — Current Code

```
// repository/index.ts — CURRENT
async create(
  orderData: {
    user_id: number;
    subTotal: number;
    order_type: OrderTypeEnum;
    transaction_fees: number;
    // ... 20 more fields inline ■
    orderDetails: Array<{
      price: number;
      total: number;
      // ... more inline types ■
    }>;
    transactions?: Array<{...}> | null;
  },
  trx?: any // ← 'any' transaction type ■
) { ... }
```

AFTER — Clean Code

```
// types/order.dto.ts — CLEAN
export interface CreateOrderDto {
  userId: number;
  subTotal: Decimal;
  orderType: OrderTypeEnum;
  transactionFees: Decimal;
  total: Decimal;
  orderDetails: CreateOrderDetailDto[];
  goldTransactions: CreateGoldTransactionDto[];
  transactions?: CreateTransactionDto[] | null;
}

// repository/index.ts — CLEAN
async create(
  orderData: CreateOrderDto,
  trx?: Knex.Transaction // ← proper type ■
): Promise<OrdersModel> { ... }
```

3.5 — MySQL-Specific Raw SQL (repository/index.ts)

9 occurrences of MySQL-only functions are spread across the repository. These will break immediately when migrating to PostgreSQL.

MySQL (current)	ANSI SQL / PostgreSQL (target)
IF(metalType='silver', amount, 0)	CASE WHEN metalType='silver' THEN amount ELSE 0 END
MONTH(created)	EXTRACT(MONTH FROM created_at)
COUNT(IF(metalType='gold', 1, NULL))	COUNT(CASE WHEN metalType='gold' THEN 1 END)
UTC_TIMESTAMP()	NOW() or CURRENT_TIMESTAMP
concat(firstName, ' ', lastName)	CONCAT(first_name, ' ', last_name)

MySQL (current)	ANSI SQL / PostgreSQL (target)
<code>CAST(SUM(amount) AS DECIMAL(15,2))</code>	<code>SUM(amount)::NUMERIC(15,2)</code> (PostgreSQL)

3.6 — Redundant Count Query (repository/index.ts)

Two separate database round-trips fetch paginated results and the total count independently. Objection's built-in `.page()` already returns both in one query.

BEFORE — Current Code

```
// CURRENT — 2 DB round-trips ■
const orders = await this.model.query()
  .where({ user_id })
  .page(page, limit); // trip 1

const { total_count } = await this.model.query()
  .count('id as total_count')
  .first()
  .where({ user_id }); // trip 2 — same data! ■
```

AFTER — Clean Code

```
// CLEAN — 1 DB round-trip ■
const { results, total } = await this.model.query()
  .where({ user_id })
  .page(page, limit); // already returns total

return {
  result:      results,
  totalItems: total,
  totalPages: Math.ceil(total / limit),
};
```

SECTION 4

Transaction Domain — Deep Dive

4.1 — Typo in Class Name (repository/index.ts)

BEFORE — Current Code

```
class TransactionRepoistory {
  // ← 'Repoistory' is misspelled ■
  // affects searchability, IDE navigation,
  // and code professionalism
```

AFTER — Clean Code

```
class TransactionRepository {
  // ← correct spelling ■
  // Also rename file from index.ts to
  // transaction.repository.ts
```

4.2 — Parameter Reassignment with var (repository/index.ts)

Re-declaring a function parameter with **var** inside the function body creates a confusing name collision and signals a misunderstanding of JavaScript scoping.

BEFORE — Current Code

```
async userTransaction(
  user_id: number,
  { lang, ... }: { lang: string; ... }
) {
  var lang = lang == 'en' ? 'en' : 'ar'
  // ← re-declares parameter with var ■
  // shadows itself – confusing scope
}
```

AFTER — Clean Code

```
async userTransaction(
  userId: number,
  { lang: rawLang, ... }: { lang: string; ... }
) {
  const lang = rawLang === 'en' ? 'en' : 'ar';
  // ← destructure rename + const ■
  // clear, unambiguous, strict-mode safe
}
```

4.3 — Fragile JSON Parsing (repository/index.ts)

Three occurrences parse the same string **twice per check** using an inconsistent fallback (`{}` is an object, not an array). If the input string is 'null' or malformed JSON, this throws an unhandled exception.

BEFORE — Current Code

```
// CURRENT – parses twice, fragile fallback ■
if (Array.isArray(JSON.parse(type || '{}'))
  && JSON.parse(type || '[]').length)
  q.whereIn('type', JSON.parse(type!))

if (Array.isArray(JSON.parse(status || '{}'))
  && JSON.parse(status || '[]').length)
  q.whereIn('status_id', JSON.parse(status!))

// Problems:
// - parses same string 3x per filter ■
// - '{}' fallback then checks isArray ■
// - no try/catch – throws on bad JSON ■
```

AFTER — Clean Code

```
// utils/parseJsonArray.ts – CLEAN ■
export function parseJsonArray<T>(raw?: string): T[] {
  if (!raw) return [];
  try {
    const parsed = JSON.parse(raw);
    return Array.isArray(parsed) ? parsed : [];
  } catch { return []; }
}

// repository – CLEAN ■
const typeFilter = parseJsonArray<number>(type);
const statusFilter = parseJsonArray<number>(status);
const metalFilter = parseJsonArray<string>(metalType);

if (typeFilter.length) q.whereIn('type', typeFilter);
if (statusFilter.length) q.whereIn('status_id', statusFilter);
if (metalFilter.length) q.whereIn('metalType', metalFilter);
```

4.4 — Cache Invalidation Inside the Data Layer (repository/index.ts)

The repository calls **redisClient.del()** after inserting. A repository's only job is data persistence. Cache invalidation is an application-layer concern and belongs in the service.

BEFORE — Current Code

```
// CURRENT — repository knows about Redis ■
class TransactionRepository {
  async create(data, trx?: any) {
    const tx = await this.model
      .query(trx).insert(data);
    redisClient.del(
      `api:transaction/currentBalance_${data.user_id}`
    ); // ■ cache concern in data layer!
    return tx;
  }
}
```

AFTER — Clean Code

```
// CLEAN — repository is pure data layer ■
class TransactionRepository {
  async create(data: CreateTransactionDto,
    trx?: Knex.Transaction) {
    return this.model.query(trx).insert(data);
    // ■ no Redis, no cache, pure persistence
  }
}

// service/deposit.ts — cache lives here ■
const tx = await transactionRepo.create({...});
await balanceCache.invalidate(dto.userId);
```

4.5 — Ambiguous Balance Field (services/currentBalance.ts)**BEFORE — Current Code**

```
// CURRENT — hidden aliasing ■
return {
  data: {
    balance: {
      availWithdraw: userBalance.availBalance,
      // ← same field, two names!
      ...userBalance, // ← spreads availBalance again
    },
  },
};
```

AFTER — Clean Code

```
// CLEAN — explicit interface ■
export interface WalletBalance {
  current: number; // accepted credits - debits
  availBalance: number; // available for any use
  availWithdraw: number; // specifically for withdrawal
  pending: number; // pending withdrawals
  pendingDeposit: number; // pending deposits
}

const balance: WalletBalance = {
  current: raw.current,
  availBalance: raw.availBalance,
  availWithdraw: raw.availBalance, // same for now
  // separate field supports future restrictions
};
```

4.6 — Ambiguous Withdraw Validation (services/withdraw.ts)**BEFORE — Current Code**

```
// CURRENT — hard to read ■
if (Number(userBalance.availWithdraw) < amount
  || userBalance.availBalance < amount)
  throw new BadRequestException(
    i18n.__mf({ phrase: '...', locale: lang }, {
      availWithdraw:
        userBalance.availWithdraw < userBalance.availBalance
        ? userBalance.availWithdraw
        : userBalance.availBalance, // Math.min
    })
  )
```

AFTER — Clean Code

```
// CLEAN — named, readable, testable ■
const withdrawable = Math.min(
  balance.availWithdraw,
  balance.availBalance
);

if (withdrawable < amount) {
  throw new BadRequestException(
    i18n.__mf(
      { phrase: 'transactions.Withdraw.insufficient',
        locale: lang },
      { availWithdraw: withdrawable.toFixed(2) }
    )
  );
}
```

SECTION 5

Clean Target Folder Structure

Order/ Domain

<code>Order/controllers/add.controller.ts</code>	Thin HTTP adapter — keep as-is
<code>Order/services/add.service.ts</code>	Orchestrator only — slim
<code>Order/services/payment/wallet.service.ts</code> ← NEW	Wallet payment path
<code>Order/services/payment/visa.service.ts</code> ← NEW	Visa payment path
<code>Order/services/payment/pending.service.ts</code> ← NEW	Pending order lifecycle
<code>Order/repository/order.repository.ts</code>	Renamed from index.ts
<code>Order/repository/pendingOrder.repository.ts</code>	Unchanged
<code>Order/calculators/calculateOrder.ts</code> ← NEW	Pure calculation — no side effects
<code>Order/calculators/applyVoucher.ts</code> ← NEW	Voucher discount logic
<code>Order/types/order.type.ts</code>	Shared enums
<code>Order/types/order.dto.ts</code> ← NEW	Input/output shapes
<code>Order/types/order.response.ts</code> ← NEW	Response shapes

Transaction/ Domain

<code>Transaction/controllers/deposit.controller.ts</code>	Thin HTTP adapter
<code>Transaction/controllers/withdraw.controller.ts</code>	Thin HTTP adapter
<code>Transaction/services/deposit.service.ts</code>	Typed DTO, no raw objects
<code>Transaction/services/withdraw.service.ts</code>	4 named validation helpers
<code>Transaction/services/currentBalance.service.ts</code>	Explicit WalletBalance interface
<code>Transaction/repository/transaction.repository.ts</code>	Fixed typo, no Redis calls
<code>Transaction/cache/balance.cache.ts</code> ← NEW	Cache logic extracted here
<code>Transaction/utils/parseJsonArray.ts</code> ← NEW	Safe JSON filter parser
<code>Transaction/types/transaction.dto.ts</code> ← NEW	DepositDto, WithdrawDto, etc.
<code>Transaction/types/transaction.response.ts</code> ← NEW	TransactionResponse shape

SECTION 6

Refactored Code Examples

6.1 — Clean depositService

```
// types/transaction.dto.ts
export interface DepositDto {
  userId: number; paymentTypeId: number;
  transactionDate: Date; amount: number;
  invoice: string; requestId: string;
}

// services/deposit.service.ts
export const depositService = async (dto: DepositDto)
  : Promise<{ data: { transaction: TransactionResponse } }> => {

  const transaction = await transactionRepo.create({
    userId: dto.userId, paymentTypeId: dto.paymentTypeId,
    type: TransactionTypeEnum.add, requestId: dto.requestId,
    amount: dto.amount, invoice: dto.invoice,
  });

  await balanceCache.invalidate(dto.userId); // cache is service concern ■
  notifyTransactionUpdate(transaction);      // wraps push + socket emit

  return { data: { transaction: toTransactionResponse(transaction) } };
};
```

6.2 — Clean withdrawService (4 named steps)

```
// services/withdraw.service.ts
export const withdrawService = async (dto: WithdrawDto)
  : Promise<{ data: { transaction: TransactionResponse } }> => {

  await validateMinWithdrawAmount(dto.amount, dto.lang); // step 1
  await validateSufficientBalance(dto.userId, dto.amount, dto.lang); // step 2
  const bank = await validateBankExists(dto.bankId);      // step 3

  const transaction = await transactionRepo.create({      // step 4
    amount: -dto.amount, paymentTypeId: PaymentTypeEnum.bankTransfer,
    bankId: bank.id, fullName: dto.fullName, iban: dto.iban,
    type: dto.type, userId: dto.userId, requestId: dto.requestId,
  });

  await balanceCache.invalidate(dto.userId);
  notifyTransactionUpdate(transaction);
  return { data: { transaction: toTransactionResponse(transaction) } };
};
```

6.3 — Clean Balance Cache (cache/balance.cache.ts)

```
// cache/balance.cache.ts
const CACHE_KEY = (userId: number) =>
  `api:transaction/currentBalance_${userId}`;
const TTL_SECONDS = 60;

export const balanceCache = {
  async get(userId: number): Promise<WalletBalance | null> {
    const raw = await redisClient.get(CACHE_KEY(userId));
    return raw ? JSON.parse(raw) : null;
  },
  async set(userId: number, balance: WalletBalance): Promise<void> {
    await redisClient.set(CACHE_KEY(userId), JSON.stringify(balance),
      { EX: TTL_SECONDS });
  },
  async invalidate(userId: number): Promise<void> {
    await redisClient.del(CACHE_KEY(userId));
  },
};
```

SECTION 7

Cross-Cutting Improvements**7.1 — Standardize Response Shape**

Problem	Every service returns a different ad-hoc object shape
Solution	ApiResponse and PaginatedApiResponse in libs/response.ts
Benefit	Frontend and test code can rely on a single predictable contract

7.2 — Standardize Pagination Parameters

Problem	pageNum vs page, PageCount (capital C), 0-indexed mixed with 1-indexed
Solution	normalizePagination() helper: page → 0-indexed, pageSize capped at 100
Benefit	Consistent API contract across all listing endpoints

7.3 — Use Proper Transaction Type

Problem	trx?: any — no type safety, no IDE autocompletion
Solution	import type { Transaction } from 'objection' → trx?: Transaction
Benefit	Compile-time safety for transaction usage in repositories

7.4 — Replace var with const / let

Problem	5 var declarations in add.service.ts — function-scoped, not block-scoped
Solution	Replace all var with const (preferred) or let
Benefit	Enable strict: true in tsconfig.json to catch this at compile time

7.5 — Remove fs.appendFileSync from Business Logic

Problem	Synchronous disk writes block Node.js event loop; data leaks to plain text
Solution	Use logger.info() / logger.debug() from core logger everywhere
Benefit	Non-blocking, structured, searchable, environment-aware logging

SECTION 8

Priority Action Plan — 3 Sprints

Sprint 1 — Critical Fixes

Day 1–2	■ Delete all console.log() calls from production code paths
	■ Delete all fs.appendFileSync() calls
	■ Replace with logger.info() / logger.debug() from core logger
Day 3–4	■ Extract cache logic to Transaction/cache/balance.cache.ts
	■ Fix TransactionRepoistory typo → TransactionRepository
	■ Replace 5× var with const/let in Order/services/add.ts
Day 5	■ Add utils/parseJsonArray.ts utility
	■ Replace all JSON.parse(x '{}') usages in Transaction repository

Sprint 2 — High-Priority Refactors

Week 2	■ Extract CreateOrderDto, CreateTransactionDto types into dto files
	■ Split addService() into: validateOrderPreconditions(), handleWalletPaymentPath(), handleVisaPaymentPath()
	■ Split withdrawService() into 3 private validation helper functions
Week 3	■ Fix balance field aliasing in currentBalance.ts — add explicit WalletBalance interface
	■ Remove all large commented-out code blocks
	■ Add clean depositService with typed DTO and toTransactionResponse() mapper

Sprint 3 — Medium-Priority Cleanup

Week 4	■ Replace MySQL IF() with CASE WHEN in all raw SQL
	■ Replace MONTH() with EXTRACT(MONTH FROM created_at)
	■ Replace UTC_TIMESTAMP() with NOW() / CURRENT_TIMESTAMP
Week 4	■ Standardize pagination: pageNum → page, PageCount → pageSize
	■ Add PaginatedApiResponse shared type in libs/response.ts
	■ Enable strict: true in tsconfig.json and fix resulting type errors

SECTION 9

Before vs After Scorecard

After all three sprints are complete, the codebase will reach the following measurable improvements:

Metric	Before	After	Impact
Longest service function	476 lines	~60 lines (orchestrator)	HIGH
Longest repository method param	50-line inline type	Named DTO types	HIGH
any type usage	8 occurrences	0	HIGH
var declarations	5 occurrences	0	MEDIUM
MySQL-specific raw SQL	9 occurrences	0	HIGH
console.log in production	1	0	CRITICAL
fs.appendFileSync in production	3	0	CRITICAL
Cache logic in repository	Yes	No (service layer)	HIGH
Shared response type	No	Yes (ApiResponse)	MEDIUM
Testable pure functions	0	4+ (parseJsonArray, aggregateBalance...)	HIGH
Dead/commented code blocks	2 large blocks	0	MEDIUM
Typos in class names	1 (TransactionRepoistory)	0	LOW
Duplicate DB count queries	Yes	No (Objection .page())	MEDIUM
trx type safety	any	Knex.Transaction	MEDIUM

All code examples follow TypeScript strict mode. Patterns are ORM-agnostic (Knex/Objection today, Prisma-compatible after migration).
Report generated on April 29, 2026.

Sabeka Backend v2.2.0 · src/app/api/Order/ · src/app/api/Transaction/